

INTRODUZIONE

Idea Fondamentale

Mappare chiavi in posizioni di array tramite **funzione di hash** $h(k)$.

Obiettivo: Accesso diretto $O(1)$ per Insert, Search, Delete.

Problema: Collisioni - due chiavi mappate nella stessa posizione.

CHAINING (Concatenamento)

Struttura

```
T[0] → lista collisioni  
T[1] → lista collisioni  
...  
T[m-1] → lista collisioni
```

Operazioni

ChainedHashInsert

```
ChainedHashInsert(T, x):  
    inserisci x in testa a T[h(x.key)]
```

Complessità: $O(1)$

ChainedHashSearch

```
ChainedHashSearch(T, k):  
    ricerca elemento con chiave k in lista T[h(k)]
```

Complessità: $O(1 + \alpha)$ atteso, dove $\alpha = n/m$ è il **fattore di carico**

ChainedHashDelete

```
ChainedHashDelete(T, x):  
    rimuovi x dalla lista T[h(x.key)]
```

Complessità: $O(1)$ se lista doppiamente collegata

Analisi

Fattore di carico: $\alpha = n/m$

- n = numero elementi
- m = dimensione tabella

Ipotesi di hashing uniforme semplice: Ogni chiave ha uguale probabilità $1/m$ di finire in qualunque slot.

Teorema:

- **Ricerca senza successo:** $\Theta(1 + \alpha)$ atteso
- **Ricerca con successo:** $\Theta(1 + \alpha)$ atteso

Conclusione: Se $\alpha = O(1)$ (cioè $n = O(m)$), tutte le operazioni sono $O(1)$ attese.

FUNZIONI DI HASH

Proprietà Desiderabili

1. **Deterministica:** $h(k)$ sempre uguale per stessa chiave k
2. **Uniforme:** distribuzione uniforme su $[0, m-1]$
3. **Efficiente:** calcolo veloce $O(1)$

Metodo della Divisione

$$h(k) = k \bmod m$$

Scelta di m :

- ✓ Buono: m primo non troppo vicino a potenza di 2
- ✗ Evitare: $m = 2^p$ (usa solo p bit meno significativi)
- ✗ Evitare: $m = 2^p - 1$ con chiavi stringhe (anagrammi collidono)

Esempio Problema:

$$m = 2^5 - 1 = 31$$

Chiavi: parole di 5 bit ciascuna

"abcde" e "bcdea" (anagrammi) $\rightarrow$ stessa somma mod 31 $\rightarrow$ COLLISIONE

Metodo della Moltiplicazione

$$h(k) = \lfloor m \cdot (k \cdot A \bmod 1) \rfloor$$

dove $A \in (0,1)$ è costante (es. $A \approx (\sqrt{5} - 1)/2 \approx 0.6180339887\dots$)

Vantaggi:

- m può essere qualunque (tipicamente $m = 2^p$)
- Meno sensibile alla scelta di m

Hashing Universale

Definizione: Famiglia H di funzioni hash è **universale** se per ogni coppia di chiavi $k_1 \neq k_2$:

$$\Pr[h(k_1) = h(k_2)] \leq 1/m$$

dove h è scelta casualmente da H .

Esempio (Metodo della Moltiplicazione Modulare):

$$H = \{h_{ab} : a, b \in \mathbb{Z}_p, a \neq 0\}$$

$$h_{ab}(k) = ((a \cdot k + b) \bmod p) \bmod m$$

dove p primo $> \max(\text{chiavi})$, m dimensione tabella.

OPEN ADDRESSING (Indirizzamento Aperto)

Idea

Tutte le chiavi memorizzate direttamente nell'array (no liste).

Collisione: Prova altre posizioni secondo **sequenza di ispezione**.

Funzione di Hash

$$h: U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\}$$

$$h(k, 0), h(k, 1), h(k, 2), \dots, h(k, m-1)$$

deve essere permutazione di $\{0, 1, \dots, m-1\}$.

Operazioni

HashInsert

```

HashInsert(T, k):
    i = 0
    repeat:
        j = h(k, i)
        if T[j] = nil:
            T[j] = k
            return j
        else:
            i = i + 1
    until i = m
    error "hash table overflow"

```

Complessità: Dipende da α e tecnica di probing

HashSearch

```

HashSearch(T, k):
    i = 0
    repeat:
        j = h(k, i)
        if T[j] = k:
            return j
        i = i + 1
    until T[j] = nil OR i = m
    return nil

```

Tecniche di Probing

1. Linear Probing

$$h(k, i) = (h'(k) + i) \bmod m$$

Vantaggi:

- Semplice
- Buona cache locality

Svantaggi:

- **Primary clustering:** tendono a formarsi lunghe sequenze di celle occupate
- Prestazioni degradano rapidamente con α alto

Esempio:

```

m = 8, h'(k) = k mod 8
Inserimento chiavi: 12, 3, 22

```

```
h(12,0) = 12 mod 8 = 4 → T[4] = 12
h(3,0) = 3 mod 8 = 3 → T[3] = 3
h(22,0) = 22 mod 8 = 6 → T[6] = 22
```

2. Quadratic Probing

$$h(k, i) = (h'(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$$

Vantaggi:

- Riduce primary clustering

Svantaggi:

- **Secondary clustering:** chiavi con stesso $h'(k)$ seguono stessa sequenza
- Richiede m potenza di 2 o primo, c_1, c_2 specifici per coprire tutte le celle

3. Double Hashing (★ Preferito)

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

Scelta funzioni:

```
h1(k) = k mod m
h2(k) = 1 + (k mod (m-2))    // IMPORTANTE: h2(k) ≠ 0 sempre
```

Vantaggi:

- $\Theta(m^2)$ sequenze di ispezione diverse
- Migliori prestazioni
- Approssima hashing uniforme

Esempio:

$m = 8$, inserimento: 12, 3, 22, 14, 38

```
h1(k) = k mod 8
h2(k) = 1 + (k mod 6)
```

```
12: h1(12)=4, h2(12)=1 → probe: 4 → T[4]=12
 3: h1(3)=3, h2(3)=4 → probe: 3 → T[3]=3
22: h1(22)=6, h2(22)=5 → probe: 6 → T[6]=22
14: h1(14)=6, h2(14)=3 → probe: 6→1 → T[1]=14
38: h1(38)=6, h2(38)=3 → probe: 6→1→4→7 → T[7]=38
```

Risultato:

$T = [-, 14, -, 3, 12, -, 22, 38]$

Analisi Open Addressing

Teorema (Hashing Uniforme):

- **Inserimento senza successo:** $\leq 1/(1-\alpha)$ probe attese
- **Ricerca con successo:** $\leq (1/\alpha) \cdot \ln(1/(1-\alpha))$ probe attese

$\alpha = n/m$ deve essere < 1 (altrimenti overflow)

Raccomandazione pratica: Mantenere $\alpha \leq 0.7$ per buone prestazioni



CONFRONTO TECNICHE

Aspetto	Chaining	Open Addressing
Spazio	$O(n+m)$	$O(m)$, $m \geq n$
α massimo	Illimitato	< 1
Implementazione	Più complessa	Più semplice
Cache performance	Peggior	Migliore
Robustezza	Migliore	Peggior (degrada con $\alpha \rightarrow 1$)
Delete	Facile	Complessa (servono marcatori)



ESERCIZI TIPICI D'ESAME

Tipo 1: Tracciare Inserimenti

Testo:

Tabella hash $m=8$, chaining, $h(k) = k \bmod m$
Inserire: 13, 10, 33, 21, 8, 26

Soluzione:

$T[0]: \rightarrow$
 $T[1]: \rightarrow 33 \rightarrow 25$
 $T[2]: \rightarrow 10$
 $T[3]: \rightarrow$

```
T[4]: →  
T[5]: → 13 → 21  
T[6]: → 22  
T[7]: →
```

Tipo 2: Double Hashing

Testo:

```
m = 9, double hashing  
h1(k) = k mod 9  
h2(k) = 1 + (k mod 7)  
Inserire: 12, 3, 22, 14, 38
```

Soluzione:

```
12: h1=3, h2=6 → 3 → T[3]=12  
3: h1=3, h2=4 → 3→7 → T[7]=3  
22: h1=4, h2=2 → 4 → T[4]=22  
14: h1=5, h2=1 → 5 → T[5]=14  
38: h1=2, h2=4 → 2 → T[2]=38  
  
T = [-, -, 38, 12, 22, 14, -, 3, -]
```

Tipo 3: Funzione Hash Invariante

Problema Anagramma: Mostrare che $h(k) = k \bmod (2^p - 1)$ è invariante rispetto a permutazioni di parole quando k è composto da w parole di p bit.

Dimostrazione:

Sia $k = w_1 \cdot 2^{((w-1)p)} + w_2 \cdot 2^{((w-2)p)} + \dots + w_w \cdot 2^0$

Poiché $2^p \equiv 1 \pmod{2^p - 1}$:

$$\begin{aligned} h(k) &= k \bmod (2^p - 1) \\ &= (w_1 + w_2 + \dots + w_w) \bmod (2^p - 1) \end{aligned}$$

Quindi $h(k)$ dipende solo dalla SOMMA delle parole, non dall'ordine.
Anagrammi (permutazioni) hanno stessa somma → COLLISIONE x

CONSIGLI PER L'ESAME

Chaining

1. Disegna sempre le liste
2. Inserimento in testa → più recente a sinistra
3. Ricorda: $\alpha = n/m$ può essere > 1

Open Addressing

1. **Double hashing:** SEMPRE $h_2(k) \neq 0$
2. Traccia TUTTE le posizioni ispezionate
3. Se $T[j]$ occupato → continua sequenza
4. $\alpha < 1$ sempre (altrimenti overflow)

Funzioni Hash

1. **Divisione:** m primo, non vicino a 2^p
2. **$m = 2^p - 1$:** evitare con stringhe (anagrammi collidono)
3. **Double hashing:** h_2 e m coprimi (es. m primo, $h_2(k) < m$)

Complessità

- **Chaining:** $O(1 + \alpha)$
- **Open Addressing:** $O(1/(1-\alpha))$
- Se $\alpha = O(1) \rightarrow$ entrambe $O(1)$

Errori Comuni

- ✗ Dimenticare che open addressing richiede $\alpha < 1$
- ✗ In double hashing, permettere $h_2(k) = 0$
- ✗ Confondere "probe" con "collisione"
- ✗ Non tracciare tutte le posizioni in open addressing